(F) Since not all the bits we checked were equals to `1`, it means that "`duffy@acme.com`".is not stored in the Bloom filter, and we will return `false`.
The same workflow is followed for "`roger@acme.com`", on the right: because all three bits checked are set to `1`, we return `true`: this means that "`roger@acme.com`" *might* have been stored in the Bloom filter, with a certain degree of confidence.

With all the helper functions we already defined, checking the Bloom filter for a key becomes a piece of cake: we just need to retrieve the positions where the bits for the key would be stored, and check that those bits are all 1. To review the whole process, you can take a look at figure 4.4, and find the pseudocode for the method in listing 4.10.

You might have noticed that in `contains` we check that the value returned by `readBit` isn't `0`: while it would technically suffice to check that the bit we read is equal to `1`, this would force us to use an extra right-shift bitwise operation. If our bit is stored in the `i`-th element of the array as its `j`-th bit (from the right), in fact, we should in theory shift the result of our bitwise extraction process `j` bits to the right, or compare it with a mask made of a single `1` shifted `j` positions to the left. This way, instead, we don't need to, and we can trim off a few milliseconds (per operation) from our implementation.

Also notice that the method takes an optional second parameter: the reason will become clear in the next section.

4.6.7 Storing a Key